INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

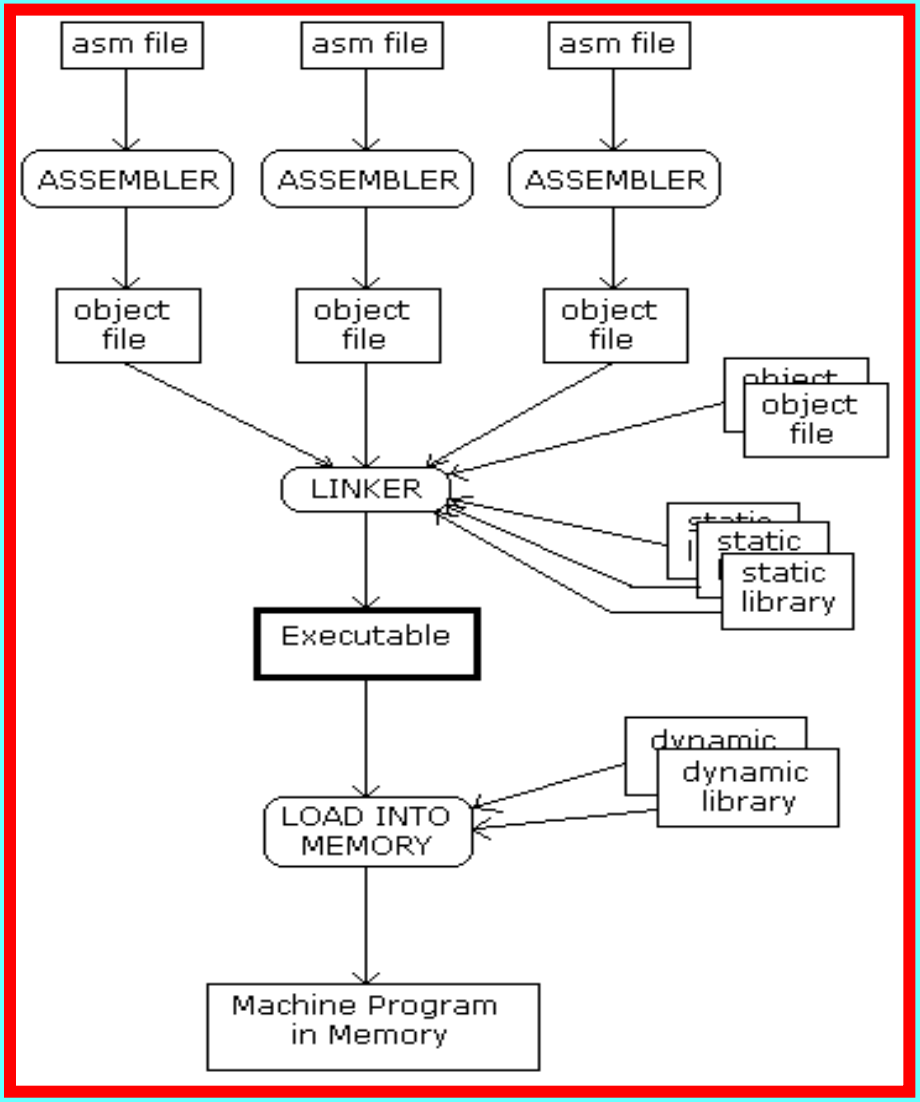
INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

WEEK6 LECTURE2

INTEL X86-64

ASSEMBLY LANGUAGE PROGRAMMING

X86-64 Assembly Language Programming



Each assembly language file is assembled into an "object file" and the object files are linked with other object files to form an executable. A "static library" is really nothing more than a collection of (probably related) object files. Application programmers generally make use of libraries for things like I/O and math.

X86-64 Assembly Language Programming

Registers

Temporary results are stored in the general-purpose registers RAX, RBX, RCX, RDX, RSI, RDI, and R8 through R15.

These registers are 64 bits wide, however different parts of the registers can be used by specifying different variations of the register names.

The 32-bit version of a register is just the low 32 bits, and the 16-bit version is the lowest 16 bits. The high and low 8-bit registers refer to the high and low parts of their 16-bit counterparts.

In addition to the registers above, which were available in 32-bit mode, 64-bit x86 adds eight new general purpose registers: R8, R9, R10, R11, R12, R13, R14, and R15.

Intended Purpose	64-bit	32-bit	16-bit	high 8-bit	low 8-bit
Accumulator	RAX	EAX	AX	AH	AL
Base index	RBX	EBX	BX	BH	BL
Counter	RCX	ECX	CX	CH	CL
Accumulator	RDX	EDX	DX	DH	DL
Source index	RSI	ESI	SI		
Destination index	RDI	EDI	DI		
Stack pointer	RSP	RSP	SP		
Stack base pointer	RBP	EBP	BP		
Instruction pointer	RIP	EIP	IP		

Operands

- Each instruction takes 0-2 operands. The following types of operands exist:
  - Register (short name = r)
  - Immediate value (constant) (short name = im)
  - Memory location (short name = m)
- Each instruction allows only some of these operand types for each of its operands.
- Instruction set references usually list which operand types are applicable for an instruction by listing the short name of the operand plus the accepted bit size(s).
  - For example r/m means either a register or memory location.

Assembler	Operand type	What it means
`$0`	immediate	decimal 0
`$0x10`	immediate	hexadecimal 10 (=16 decimal)
`lbl`	memory location	value stored at address of label `lbl`
`lbl+2`	memory location	value stored at two bytes after label `lbl`
`$lbl`	immediate	address of label `lbl`
`$(lbl+4)`	immediate	address of label `lbl` plus 4
`%rdx`	register	value stored in `RDX`
`(%rax)`	memory location	value at the address stored in `RAX`
`8(%rbp)`	memory location	value at eight bytes after the address stored in `RBP`
`-3(%rax)`	memory location	value at three bytes before the address stored in `RAX`

Instruction Naming

Instruction names are derived from the base instruction mnemonic, and a suffix indicating the operand sizes. For example, a MOV with quad word operands has the name movq in GNU AS.

GNU AS Instruction suffixes:

Suffix	Operand size
b	byte, 8 bits
w	word, 16 bit
l	long, 32 bit integer
q	quad word, 64 bit

Using incorrect operands with an instruction will result in the assembler complaining. For example the line movq %ebx, %eax will produce the following error message: **"Error: operand type mismatch for movq"** because it is not allowed to use 32-bit operands with a 64-bit MOV. Unfortunately the assembler does not say which operand types it expected.

X86-64 Assembly Language Programming

Instruction Listing

Below is a listing of a set of useful instructions, for quick reference. This listing follows the **AT&T convention** that the first operand is the source (src), and the second is the destination operand (dest).

Remember that to use 64-bit operands with an instruction you must use the 'q' suffix, e.g. negq %rax.

Instruction < Mnemonic – Description >	Operands *src*	Operands *dest*	Operation
ADD – Add	r/m/im	r/m	$dest \leftarrow dest + src$
AND – Bitwise logical AND	r/m/im	r/m	$dest \leftarrow AND(dest, src)$
CALL – Call procedure		r/m/im	push RIP, then $RIP \leftarrow dest$
CMP – Compare two operands	r/m/im	r/m	modify status flags similar to SUB
DEC – Decrement by 1		r/m	$dest \leftarrow dest - 1$
IDIV – Signed divide	r/m		signed divide $RDX : RAX$ by src $RAX \leftarrow quotient$, $RDX \leftarrow remainder$
IMUL – Signed multiply (2 op)	r/m/im	r	$dest \leftarrow dest * src$
IMUL – Signed multiply (1 op)	r/m		$RDX : RAX \leftarrow RAX * src$
INC – Increment by 1		r/m	$dest \leftarrow dest + 1$
Jcc – Jump if condition is met		m/im	conditionally $RIP \leftarrow dest$
JMP – Unconditional jump		m/im	$RIP \leftarrow dest$
LEA – Load effective address	m	r	$dest \leftarrow addressOf(src)$
MOV – Move	r/m/im	r/m	$dest \leftarrow src$
NEG – Two's Complement negation		r/m	$dest \leftarrow -dest$
NOT – One's Complement negation		r/m	$dest \leftarrow NOT(dest)$
OR – Bitwise logical OR	r/m/im	r/m	$dest \leftarrow OR(dest, src)$
POP – Pop value off the stack		r/m	$dest \leftarrow POP(stack)$
PUSH – Push value on the stack	r/m/im		$PUSH(stack, src)$
RET – Return from procedure			restore RIP by popping the stack
SUB – Subtract	r/m/im	r/m	$dest \leftarrow dest - src$
SYSCALL – System Call			invoke OS kernel

X86-64 Assembly Language Programming

There are many variations of the conditional jump instruction, Jcc. Here are a few: The CMP instruction is responsible for updating the status flags used by the conditional jump instructions.

For example, the JGE instruction jumps to the given target if the dest operand of the previous CMP was greater than the src operand.

```
cmp %rbx, %rax          %rax - %rbx
jge  DOWN               if %rax >= %rbx then goto DOWN
.....
.....
DOWN: sub %rbx, %rax    %rax = %rax - %rbx
```

JMP – UNCONDITIONAL JUMP INSTRUCTION

Instruction	Description
JE	Jump if equal
JNE	Jump if not equal
JG	Jump if greater than
JGE	Jump if greater than or equal
JL	Jump if less than
JLE	Jump if less than or equal

After Unsigned integer comparison, use the following conditional branches

JA - Jump Above similar to JG after signed comparison

JB - Jump below Similar to JL after signed comparison

Linux System Calls

The simplest way to interact with the OS, and thereby use input/output, is to use the SYSCALL instruction. (syscall invokes int 0x80 or int 128)

Different Linux system functions are selected based on the value of the RAX register.

Arguments to the system call are placed in the RDI, RSI, and RDX registers. Here is a short list of some system calls and corresponding register assignments:

RAX	Function	RDI	RSI	RDX
0	sys_read	file descriptor	byte buffer address	buffer size
1	sys_write	file descriptor	byte buffer address	buffer size
60	sys_exit	error code		

A complete list of available x86-64 Linux system calls is available at **http://blog.rchapman.org/post/36801038863/linux-system-call-table-for-x86-64.** Note that 32-bit system calls work differently; this list is specifically for 64-bit assembly.

X86-64 Assembly Language Programming

$vi helloasm.s or $gedit helloasm.s

```
#program helloasm.s

        .global _start
        .data
        message: .ascii "Hello World\n"
        .text
_start:
        movq $1,%rax            #system call 1 is write
        movq $1,%rdi            #file handle 1 is stdout
        movq $message,%rsi      #address of string in memory to output
        movq $13, %rdx          #number of bytes
        syscall                 #invoke operating system to print

        movq $60, %rax          #system call 60 is exit
        xorq %rdi, %rdi         #return code 0
        syscall                 #invoke operating system to exit
```

To create object file using GNU assembler as
$as -gstabs helloasm.s -o helloasm.o
To create an executable file after linking
$ld helloasm.o -o helloasm
To execute helloasm
$./helloasm
Hello World
To call GNU Debugger
$ddd helloasm&

syscall - System Call is an explicit request made to the kernel to get a specific service from the operating system via interrupt

X86-64 Assembly Language Programming

The following program just reads one line from stdin, using sys_read, then echoes it to stdout: rwstring.s

```
        .global _start
        .data
        buf: .skip 1024
        .text
_start:
        movq $0, %rdi          # stdin file descriptor
        movq $buf, %rsi        # buffer
        movq $40, %rdx         # buffer length
        movq $0, %rax          # sys_read
        syscall

        movq $1, %rdi          # stdout file descriptor
        movq $buf, %rsi        # message to print
        movq %rax, %rdx        # message length
        movq $1, %rax          # sys_write
        syscall

        movq $0, %rdi          # exit with return code =0
        movq $60, %rax
        syscall                #sys_exit
```

To create object file using GNU assembler as
$as -gstabs rwstring.s -o rwstring.o
To create an executable file after linking
$ld rwstring.o -o rwstring
To execute rwstring
$./rwstring

The sys_read function returns the number of bytes that were read in the RAX register, so we use that as the message length in the call to sys_write.

MACHINE-LEVEL PROGRAMMING : CONTROL STRUCTURES

Jump Instructions

- **jX Instructions** (where X= e, ne, s, ns, g, ge, l, le, a, b, ae, be)
  - Jump to different part of code depending on condition codes

  e.g. jne up ← if ZF=0 then jump to instruction having the label up

jX	Condition	Description
jmp	1	Unconditional
Je,Jz	ZF	Equal / Zero
Jne, Jnz	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF) &~ZF	Greater (Signed)
jge	~(SF^OF) | ZF	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF) | ZF	Less or Equal (Signed)
ja	~CF&~ZF	Above (unsigned)
jb	CF	Below (unsigned)
jae	~CF | ZF	Above or equal (unsigned >=)
jbe	CF | ZF	Below or equal (unsigned <=)

X86-64 Assembly Language Programming

There are many variations of the conditional jump instruction, Jcc. Here are a few: The CMP instruction is responsible for updating the status flags used by the conditional jump instructions.

For example, the JGE instruction jumps to the given target if the dest operand of the previous CMP was greater than the src operand.

```
cmp %rbx, %rax          %rax - %rbx
jge  DOWN               if %rax >= %rbx then goto DOWN
.....
.....
DOWN: sub %rbx, %rax    %rax = %rax - %rbx
```

JMP – UNCONDITIONAL JUMP INSTRUCTION

Instruction	Description
JE	Jump if equal
JNE	Jump if not equal
JG	Jump if greater than
JGE	Jump if greater than or equal
JL	Jump if less than
JLE	Jump if less than or equal

After Unsigned integer comparison, use the following conditional branches

JA - Jump Above similar to JG after signed comparison

JB - Jump below Similar to JL after signed comparison

Conditional Branch Example (Old Style)

```
long absdiff
   (long x, long y)
{
   long result;
   if (x > y)
      result = x-y;
   else
      result = y-x;
   return result;
}
```

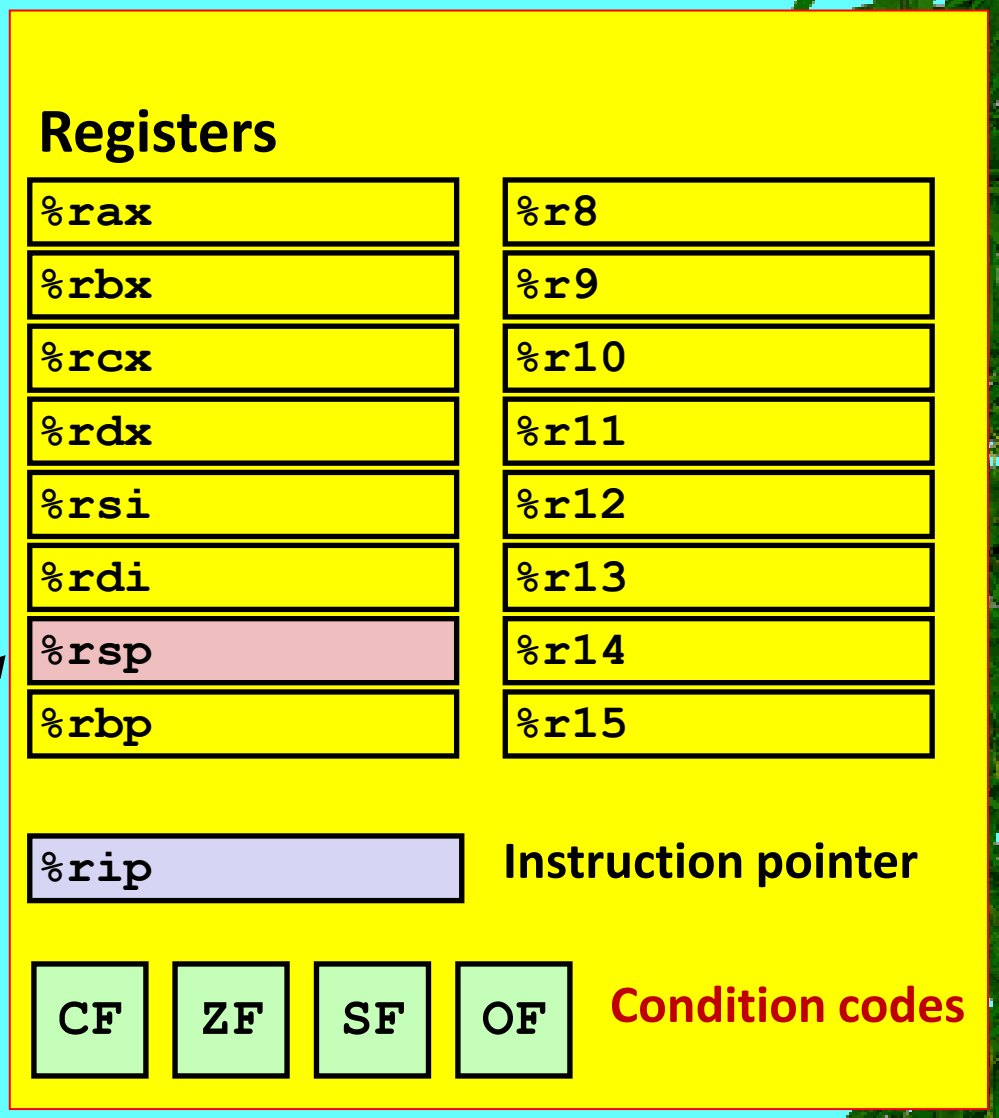
Register	Use(s)
%rdi	Argument ***x***
%rsi	Argument ***y***
%rax	Return value

```
absdiff:
    cmpq    %rsi, %rdi   # x:y
    jle     .L4     # if x<=y jump to .L$
    movq    %rdi, %rax
    subq    %rsi, %rax
    jmp .L5
.L4:            # x <= y
        movq    %rsi, %rax
        subq    %rdi, %rax
.L5:    ret
```

Processor State (x86-64, Partial)

- **Information about currently executing program**
  - Temporary data(**`%rax`**, …)
  - Location of runtime stack(**`%rsp`**)
  - Location of current code control point(**`%rip`**, …)
  - Status of recent tests (**CF, ZF, SF, OF**)

Current stack top



Condition Codes (Implicit Setting)

Condition Codes - Single bit registers

- **CF** Carry Flag (for unsigned)
- **ZF** Zero Flag
- **SF** Sign Flag (for signed)
- **OF** Overflow Flag (for signed)

- Implicitly set by arithmetic operations

Example: `addq` *Src*,*Dest*

```
t = a+b
```

CF set if carry out from most significant bit (unsigned overflow)

ZF set if `t == 0`

SF set if `t < 0` (as signed)

OF set if two's-complement (signed) overflow

```
(a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)
```

- Not set by `leaq` instruction

Condition Codes (Explicit Setting: Compare)

- **Explicit Setting by Compare Instruction**

  `cmpq` *Src2*, *Src1* *Src1 - Src2 or Dst - Src*

  `cmpq b,a` like computing **`a-b`** without setting destination

- **CF set** if carry out from most significant bit (used for unsigned comparisons)
- **ZF set** if **`a == b`**
- **SF set** if **`(a-b) < 0`** (as signed)
- **OF set** if two's-complement (signed) overflow

```
(a>0 && b<0 && (a-b)<0) ||
(a<0 && b>0 && (a-b)>0)
```

Condition Codes (Explicit Setting: Test)

- **Explicit Setting by Test instruction**

  `testq` *Src2*, *Src1* *Src1 & Src2*

  `testq b,a` like computing **`a & b`** without setting destination

- Sets condition codes based on value of *Src1 & Src2*
- Useful to have one of the operands be a mask
- **ZF set** when **`a&b == 0`**
- **SF set** when **`a&b < 0`**

```
Example :
   testq  %rax,%rax   ; ZF=1 if %rax=0
                      ; SF=1 if %rax<0
```

Accessing Condition Codes

- **Comparison and Test Instructions**

- **These instructions set the condition codes without updating any other registers**

Instruction	operation	Description
cmp src, dst	**dst - src**	**compare**
cmpb	cmpb %bl, %al	**%al - %bl; compare byte**
cmpw	cmpw %bx, %ax	**%ax - %bx; compare word**
cmpl	cmpl %ebx, %eax	**%eax - %ebx; compare long word**
cmpq	cmpq %rbx, %rax	**%rax - %rbx; compare quad word**
test src,dst	**src & dst**	**test**
testb	testb %bl, %al	**%al & %bl; test byte**
testw	testw %bx, %ax	**%ax & %bx; test word**
testl	testl %ebx, %eax	**%eax & %ebx; test long word**
testq	testq %rbx, %rax	**%rax & %rbx; test quad word**

Accessing Condition Codes

- **SetX** Instructions **(X= e, ne, s, ns, g, ge, l, le, a, b, ae, be)**
  - Set low-order byte of destination to 0 or 1 based on combinations of condition codes
  - Does not alter remaining 7 bytes

  e.g. sete %al → %al =1 if ZF=1 otherwise %al = 0

SetX	Condition	Description
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~(SF^OF) &~ZF	Greater (Signed)
setge	~(SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF) | ZF	Less or Equal (Signed)
seta	~CF&~ZF	Above (unsigned)
setb	CF	Below (unsigned)
setae	~CF	Above or equal (unsigned >=)
setbe	CF | ZF	Below or equal (unsigned <=)

Accessing Condition Codes

- **Example**

```
int  compless(int a, int b)
{
        if (a < b)  return 1;  else return 0;
}
```

Register Mapping : a in %rdi, b in %rsi, return value in %rax

```
compless :
    cmpq   %rsi,%rdi      ; compare %rdi, %rsi
    setl   %al            ; set lower byte of %rax to 0 or 1
                          ;  if %rdi < %rsi (set less than)
    movzbq  %al, %rax     ; Clear rest of %rax
    ret
```

Setting Condition Codes (Cont.)

- SetX Instructions:
  - Set single byte based on combination of condition codes
- One of addressable byte registers
  - Does not alter remaining bytes
  - Typically use **movzbq** to finish job
    - 64-bit instructions set upper 7 bytesto 0

```
int gt (long x, long y)
{
  return x > y;
}
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

```
cmpq    %rsi, %rdi    # Compare x:y
setg    %al           # Set when >
movzbq  %al, %rax     # Zero rest of %rax
ret
```

Jump Instructions

- **jX Instructions** (where X= e, ne, s, ns, g, ge, l, le, a, b, ae, be)
  - Jump to different part of code depending on condition codes

e.g. jne up ← if ZF=0 then jump to instruction having the label up

jX	Condition	Description
jmp	1	Unconditional
je	ZF	Equal / Zero
jne	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF) &~ZF	Greater (Signed)
jge	~(SF^OF)	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF) | ZF	Less or Equal (Signed)
ja	~CF&~ZF	Above (unsigned)
jb	CF	Below (unsigned)
jae	~CF	Above or equal (unsigned >=)
jbe	CF | ZF	Below or equal (unsigned <=)

Conditional Branch Example (Old Style)

```
long absdiff
   (long x, long y)
{
   long result;
   if (x > y)
      result = x-y;
   else
      result = y-x;
   return result;
}
```

Register	Use(s)
%rdi	Argument ***x***
%rsi	Argument ***y***
%rax	Return value

```
absdiff:
    cmpq    %rsi, %rdi   # x:y
    jle     .L4     # if x<=y jump to .L$
    movq    %rdi, %rax
    subq    %rsi, %rax
    ret
.L4:                # x <= y
    movq    %rsi, %rax
    subq    %rdi, %rax
    ret
```

Conditional Branch Example (Old Style)

```
long absdiff
   (long x, long y)
{
   long result;
   if (x > y)
      result = x-y;
   else
      result = y-x;
   return result;
}
```

Register	Use(s)
%rdi	Argument ***x***
%rsi	Argument ***y***
%rax	Return value

```
absdiff:
    cmpq    %rsi, %rdi   # x:y
    jle     .L4     # if x<=y jump to .L$
    movq    %rdi, %rax
    subq    %rsi, %rax
    jmp .L5
.L4:            # x <= y
        movq    %rsi, %rax
        subq    %rdi, %rax
.L5:    ret
```

X86-64 Assembly Language Programming

The following program takes each character in the string stored at message in memory and converts it to its next ascii character stores it in buffer memory at nxtcharm : nextchar.s

```
        .global _start
        .data
        message: .asciz "GOOD MORNING-How are you?\n"
        nxtcharm: .space 30

        .text
_start: movq $1, %rax
        movq $1, %rdi
        movq $message, %rsi
        movq $26, %rdx
        syscall

        movq $25, %rcx
        movq $nxtcharm, %rdi
        movq $message, %rsi
up:     movb (%rsi), %al
        addb  $1, %al
         movb %al, (%rdi)
        incq %rsi
        incq %rdi
        decq %rcx
        jnz up
```

```
movb (%rsi), %al
movb  %al, (%rdi)
incq %rsi
incq %rdi
movb (%rsi), %al
movb  %al, (%rdi)

movq $1, %rax
movq $1, %rdi
movq $nxtcharm, %rsi
movq $27, %rdx
syscall

movq $60, %rax
xorq %rdi, %rdi
syscall
```

To create object file using GNU assembler as

```
$as  -gstabs  nextchar.s   -o nextchar.o
```

To create an executable file after linking

```
$ld  nextchar.o   -o nextchar
```

To execute nextchar

```
$./nextchar
```

X86-64 Assembly Language Programming

The following program reads a string from keyboard, stores it in memory at location givenstr and then takes each character from that location and converts it to its next ascii character stores it in buffer nxtchars: rwnextchar.s

To create object file using GNU assembler as
$as -gstabs rwnextchar.s -o rwnextchar.o
To create an executable file after linking
$ld rwnextchar.o -o rwnextchar
To execute nextchar
$./rwnextchar

```
        .global _start
        .data
        message: .asciz “ENTER A STRING :"
         givenstr:  .skip 30
         nxtchars: .space 30

        .text
_start: movq $1, %rax          #sys_write
        movq $1, %rdi
        movq $message, %rsi
        movq $16, %rdx
        syscall
        movq $0, %rdi          #sys_read
        movq $givenstr, %rsi
        movq $30, %rdx
        movq $0, %rax
        syscall
        movq %rax, %r8
        movq  %r8, %rcx
        decq %rcx
        movq $givenstr, %rsi
        movq $nxtchars, %rdi
up:     movb (%rsi), %al
        addb  $1, %al
        movb %al, (%rdi)
        incq %rsi
        incq %rdi
        decq %rcx
        jnz up
        movb (%rsi), %al
        movb  %al, (%rdi)

        movq $1, %rax          #sys_write
        movq $1, %rdi
        movq $nxtchars, %rsi
        movq $r8, %rdx
        syscall

        movq $60, %rax         #sys_exit
        xorq %rdi, %rdi
        syscall
```

X86-64 Assembly Language Programming

The following program reads a string from keyboard, stores it in memory starting at location givenstr and then takes each uppercase alphabet in that string and converts it to its lowercase alphabet and stores it in memory at location lowerstr : upper2lower.s

```
        .global _start
        .data
        message: .asciz “ENTER A STRING :"
         givenstr:  .skip 100
         lowerstr: .space 100
        .text
_start: movq $1, %rax          #sys_write
        movq $1, %rdi
        movq $message, %rsi
        movq $16, %rdx
        syscall
        movq $0, %rdi           #sys_read
        movq $givenstr, %rsi
        movq $100, %rdx
        movq $0, %rax
        syscall
        movq %rax, %r8
        movq  %r8, %rcx
        decq %rcx
        movq $givenstr, %rsi
        movq $lowerstr, %rdi
up:     movb (%rsi), %al
        cmp $65, %al
        jl  down1
        cmp  $90, %al
        jg   down1
        addb  $32, %al
down1:  movb %al, (%rdi)
        incq %rsi
        incq %rdi
        decq %rcx
        jnz up
        movb (%rsi), %al
        movb  %al, (%rdi)

        movq $1, %rax
        movq $1, %rdi
        movq $lowerstr, %rsi
        movq $r8, %rdx
        syscall
        movq $60, %rax
        xorq %rdi, %rdi
        syscall
```

To create object file using GNU assembler as
$as -gstabs upper2lower.s -o upper2lower.o
To create an executable file after linking
$ld upper2lower.o   -o upper2lower
To execute upper2lower
$./upper2lower

GNU as Assembler Directives

.ascii "string"...

.ascii expects zero or more string literals separated by commas. It assembles each string (with no automatic trailing zero byte) into consecutive addresses

.asciz "string"...

.asciz is just like .ascii, but each string is followed by a zero byte. The "z" in `.asciz' stands for "zero".

.string "str"

Copy the characters in str to the object file. You may specify more than one string to copy, separated by commas. Unless otherwise specified for a particular machine, the assembler marks the end of each string with a 0 byte.

.byte expressions

.byte expects zero or more expressions, separated by commas. Each expression is assembled into the next byte.

.single flonums

This directive assembles zero or more flonums, separated by commas. It has the same effect as .float. The exact kind of floating point numbers emitted depends on how as is configured.

.float flonums

This directive assembles zero or more flonums, separated by commas. It has the same effect as .single.

The exact kind of floating point numbers emitted depends on how as is configured.

.double flonums

.double expects zero or more flonums, separated by commas. It assembles floating point numbers. The exact kind of floating point numbers emitted depends on how as is configured.

GNU **as** Assembler Directives

.quad bignums

.quad expects zero or more bignums, separated by commas. For each bignum, it emits an 8-byte integer. If the bignum won't fit in 8 bytes, it prints a warning message; and just takes the lowest order 8 bytes of the bignum. The term "quad" comes from contexts in which a "word" is two bytes; hence quad-word for 8 bytes.

.hword expressions

This expects zero or more expressions, and emits a 16 bit number for each. This directive is a synonym for `.short'; depending on the target architecture, it may also be a synonym for `.word'.

.word expressions

This directive expects zero or more expressions, of any section, separated by commas. The size of the number emitted, and its byte order, depend on what target computer the assembly is for.

.int expressions

Expect zero or more expressions, of any section, separated by commas. For each expression, emit a number that, at run time, is the value of that expression. The byte order and bit size of the number depends on what kind of target the assembly is for.

.long expressions

.long is the same as `.int', see section .int.

.short expressions

.short is normally the same as `.word'. In some configurations, however, .short and .word generate numbers of different lengths.

X86-64 Assembly Language Programming

GNU **as** Assembler Directives

.skip size , fill

This directive emits size bytes, each of value fill. Both size and fill are absolute expressions. If the comma and fill are omitted, fill is assumed to be zero. This is the same as `.space'.

.space size , fill

This directive emits size bytes, each of value fill. Both size and fill are absolute expressions. If the comma and fill are omitted, fill is assumed to be zero. This is the same as `.skip'.

.print string

as will print string on the standard output during assembly. You must put string in double quotes.

.align abs-expr, abs-expr, abs-expr

Pad the location counter (in the current subsection) to a particular storage boundary.

The first expression (which must be absolute) is the alignment required. The second expression (also absolute) gives the fill value to be stored in the padding bytes. It (and the comma) may be omitted. If it is omitted, the padding bytes are normally zero. However, on some systems, if the section is marked as containing code and the fill value is omitted, the space is filled with no-op instructions. The third expression is also absolute, and is also optional. If it is present, it is the maximum number of bytes that should be skipped by this alignment directive. If doing the alignment would require skipping more bytes than the specified maximum, then the alignment is not done at all. You can omit the fill value (the second argument) entirely by simply using two commas after the required alignment; this can be useful if you want the alignment to be filled with no-op instructions when appropriate.

The way the required alignment is specified varies from system to system. For the a29k, hppa, m68k, m88k, w65, sparc, and Hitachi SH, and i386 using ELF format, the first expression is the alignment request in bytes.

For example `.align 8' advances the location counter until it is a multiple of 8. If the location counter is already a multiple of 8, no change is needed.

X86-64 Assembly Language Programming

```
#fib.s
# -----------------------------------------------------------------------
# A 64-bit Linux application that writes the first 90 Fibonacci numbers.
# fib(n) = 0,1,1,2,3,5,8,……. i.e., fib(n)=fib(n-1)+fib(n-2), fib(2)=fib(0)+fib(1)=0+1=1
# It  needs to be linked with a C library.
#To create object file using GNU assembler as
$as  -gstabs  fib.s   -o  fib.o
To create an executable file after linking printf C library routine at runtime
$ld  -dynamic-linker  /lib64/ld-linux-x86-64.so.2  fib.o   -o fib  -lc
To execute fib                $./fib
# -----------------------------------------------------------------------
        .global _start
        .data
format:     .asciz  "%20ld\n“       # format for printf(format, arguments)
        .text
_start:
        pushq    %rbx                # we have to save this since we use it

        movq     $90, %rcx           # ecx will countdown to 0
        xorq     %rax, %rax          # rax will hold the current number
        xorq     %rbx, %rbx          # rbx will hold the next number
        incq     %rbx                # rbx is originally 1
```

printf(format, arg);
format →%rdi, arg → %rsi, 0 → %rax

X86-64 Assembly Language Programming

```
# We need to call printf, but we are using rax, rbx, and rcx.  Printf may destroy rax and rcx so we will
save these before the call and  restore them afterwards.

print:  pushq   %rax                # caller-save register
        pushq   %rcx                # caller-save register
        movq    $format, %rdi       # set 1st parameter (format)
        movq    %rax, %rsi          # set 2nd parameter (current_number)
        xorq    %rax, %rax          # because printf is varargs
                                    # Stack is already aligned because we pushed three 8 byte registers
        call    printf              # printf(format, current_number)
        popq    %rcx                # restore caller-save register
        popq    %rax                # restore caller-save register
        movq    %rax, %rdx          # save the current number
        movq    %rbx, %rax          # next number is now current
        addq    %rdx, %rbx          # get the new next number
        decq    %rcx                # count down
        jnz     print               # if not done counting, do some more
        popq    %rbx                # restore rbx before returning

        movq    $60, %rax
        xorq    %rdi, %rdi
        syscall
```

printf(format, arg);
format →%rdi, arg → %rsi, 0 → %rax

X86-64 Assembly Language Programming

```
#fibio.s
# -----------------------------------------------------------------------
# A 64-bit Linux application that writes the first n Fibonacci numbers.
# The input value n is to be read from the keyboard.
# Functions called : puts, scanf, printf
# This routine  needs to be linked with  C library functions
#To create object file using GNU assembler as
$as  -gstabs  fibio.s   -o  fibio.o
To create an executable file after linking
$ld  -dynamic-linker  /lib64/ld-linux-x86-64.so.2  fibio.o   -o fibio  -lc
To execute fibio
$./fibio
#-----------------------------------------------------------------------
        .global _start
       .data
        message:     .asciz  "ENTER A VALUE FOR n :"        # asciz puts a 0 byte at the end
        format:      .asciz  "%20ld\n"
        format1:      .asciz  "THE LIST OF FIRST n=%8ld FIBONACCI NUMBERS\n"
        f:            .string  "%d"
        x:            .quad  0
        .text
_start:
pushq    %rbx                  #  save this register content since we will use it
movq    $message, %rdi         # First messaage address (or pointer) parameter in %rdi
call    puts                  # puts(message)
```

scanf(format, &x);
format →%rdi, &x → %rsi, 0 → %rax

printf(format, arg);
format →%rdi, arg → %rsi, 0 → %rax

puts(message pointer);
message pointer → %rdi

X86-64 Assembly Language Programming

```
#READ VALUE OF n  FROM KEYBOARD  AND STORE IT IN  MEMORY LOCATION x

movq $0, %rax
movq $f, %rdi          # put scanf 1st parameter (format f -  see .data section) in %rdi
movq $x, %rsi          # put scanf 2nd  parameter (pointer to location x -  see .data section) in %rsi
                       # value n read from keyboard will be stored in location x
call scanf             # scanf(f, pointer x)
```

scanf(format, &x);
format →%rdi, &x → %rsi, 0 → %rax

```
    # PRINT VALUE OF n in x  on the screen
    movq    x, %rax                 #  %rax ←n , i.e.  Contents of memory location x
    pushq   %rax                    # caller-save register
    pushq   %rcx                    # caller-save register
    movq    $format1, %rdi          # put printf  1st parameter (format1 -  see .data section) in %rdi
    movq    %rax, %rsi              # put printf  2nd parameter ( n ) in %rsi, n   is the value to be printed
    xorq    %rax, %rax
    call    printf                  # printf(format1, current_number)
    pop     %rcx                    # restore caller-save register
    pop     %rax                    # restore caller-save register
    #COMPUTING   FIBONACCI NUMBERS
    movq    %rax, %rcx              # rcx will countdown to 0
    xorq    %rax, %rax              # rax will hold the current number
    xorq    %rbx, %rbx              # rbx will hold the next number
    incq    %rbx                    # rbx is originally 1
```

printf(format, arg);
format →%rdi, arg → %rsi, 0 → %rax

X86-64 Assembly Language Programming

```
# We need to call printf, but we are using rax, rbx, and rcx.  printf may destroy rax and
  # rcx so we will save these before the call and restore them afterwards
print:
push    %rax                    # caller-save register
push    %rcx                    # caller-save register
movq    $format, %rdi           # put printf  1st parameter (format1 -  see .data section) in %rdi
movq    %rax, %rsi              # put printf  2nd parameter (current_number - currently generated Fibonacci number ) in
                                #  %rsi;  currently generated fibonacci number is the value to be printed
xorq    %rax, %rax
call    printf                  # printf(format, current_number)
pop     %rcx                    # restore caller-save register
pop     %rax                    # restore caller-save register

movq    %rax, %rdx              # save the current number
movq    %rbx, %rax              # next number is now current
addq    %rdx, %rbx              # get the new next number
decq    %rcx                    # count down
jnz     print                   # if not done counting, do some more
pop     %rbx                    # restore rbx before returning to operating system

movq    $60, %rax               # syscall to return 0
xorq    %rdi, %rdi
syscall
```

Machine Programming I: Summary

- History of Intel processors and architectures
  - Evolutionary design leads to many quirks and artifacts
- C, assembly, machine code
  - New forms of visible state: program counter, registers, ...
  - Compiler must transform statements, expressions, procedures into low-level instruction sequences
- Assembly Basics: Registers, operands, move
  - The x86-64 move instructions cover wide range of data movement forms
- Arithmetic
  - C compiler will figure out different instruction combinations to carry out computation

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 3 of Text Book

- **Computer Systems : A Programmer's Perspective**
  - **Randal E. Bryant and David R. O'Hallaron 3rd Edition, Global Edition Prentice Hall 2016 ISBN 10:1-292-10176-8**

SOC 2040
SYSTEM PROGRAMMING

END OF

MACHINE-LEVEL PROGRAMMING I

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG